

Back To Practice

< Q1 >

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the

Select Language : C (GCC 9.2.0)

```

1 #include <stdio.h>
2
3 int main() {
4     int n, m;
5
6     printf("Enter number of processes: ");
7     scanf("%d", &n);
8     printf("\n");
9
10    printf("Enter number of resource types: ");
11    scanf("%d", &m);
12    printf("\n");
13
14    int alloc[n][m], max[n][m], need[n][m];
15    int avail[m];
16
17    // Allocation
18    printf("Enter Allocation Matrix:\n");
19    for(int i = 0; i < n; i++) {
20        for(int j = 0; j < m; j++) {
21            scanf("%d", &alloc[i][j]);
22        }
23    }
24
25    // Max
26    printf("Enter Max Matrix:\n");
27    for(int i = 0; i < n; i++) {

```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the

Select Language : C (GCC 9.2.0)

```

27     for(int i = 0; i < n; i++) {
28         for(int j = 0; j < m; j++) {
29             scanf("%d", &max[i][j]);
30         }
31     }
32
33     // Available
34     printf("Enter Available Resources: ");
35     for(int j = 0; j < m; j++) {
36         scanf("%d", &avail[j]);
37     }
38
39     // Need = Max - Allocation
40     for(int i = 0; i < n; i++) {
41         for(int j = 0; j < m; j++) {
42             need[i][j] = max[i][j] - alloc[i][j];
43         }
44     }
45
46     int finish[n];
47     for(int i = 0; i < n; i++) finish[i] = 0;
48
49     int safeSeq[n];
50     int work[m];
51
52     // Work = Available
53     for(int j = 0; j < m; j++) {

```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the

Select Language : C (GCC 9.2.0)

```

53  for(int j = 0; j < m; j++) {
54      work[j] = avail[j];
55  }
56
57  int count = 0;
58
59  while(count < n) {
60      int found = 0;
61
62      for(int i = 0; i < n; i++) {
63          if(finish[i] == 0) {
64              int canExecute = 1;
65
66              for(int j = 0; j < m; j++) {
67                  if(need[i][j] > work[j]) {
68                      canExecute = 0;
69                      break;
70              }
71          }
72
73          if(canExecute) {
74              // Add allocation to work
75              for(int j = 0; j < m; j++) {
76                  work[j] += alloc[i][j];
77              }
78
79              safeSeq[count++] = i;

```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

< Q1 >

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the

Select Language : C (GCC 9.2.0)

```
73         if(canExecute) {
74             // Add allocation to work
75             for(int j = 0; j < m; j++) {
76                 work[j] += alloc[i][j];
77             }
78
79             safeSeq[count++] = i;
80             finish[i] = 1;
81             found = 1;
82         }
83     }
84 }
85
86 if(found == 0) {
87     printf("\nSystem is NOT in a safe state (Deadlock may occur).\n");
88     return 0;
89 }
90
91 printf("\nSystem is in a SAFE STATE.\nSafe Sequence: ");
92 for(int i = 0; i < n; i++) {
93     printf("P%d ", safeSeq[i]);
94 }
95 printf("\n");
96
97 return 0;
98 }
99 }
```

Test Cases 2

Run Sample Cases

Run All Cases